

A Lightweight Long-Term Memory Agent with Update and Reflection

Xinkai Ji and Jiajie Zhang and Wenxi Zhang

Nanjing University

Abstract

Long-term conversational memory is essential for dialogue agents to maintain coherent interactions across multiple sessions. Existing approaches suffer from key limitations: no-memory agents forget all prior context; full-context agents incur prohibitive token costs and latency; and raw-turn retrieval-augmented generation (RAG) introduces noise from unstructured dialogue turns. We propose a lightweight memory agent that extracts structured memories from raw dialogue, applies deduplication and merging via an update mechanism, and generates high-level reflections over active memories. On a 40-question benchmark spanning four categories (temporal, open-domain, single-hop, multi-hop), our Update+Reflection agent achieves a Judge Score of 0.325 compared to 0.263 for raw-turn RAG, while answering $6\times$ faster than full-context (1.22s vs. 7.31s). Ablation studies confirm that detail notes and memory updating are the most impactful components, while reflection yields marginal gains on fact-heavy queries. Failure analysis reveals multi-hop reasoning as the primary bottleneck, suggesting directions for future improvements.

1 Introduction

As conversational AI systems are deployed over increasingly long time horizons—spanning days, weeks, or months of user interaction—the ability to remember and leverage past conversations becomes critical. A dialogue agent that forgets a user’s preferences, life events, or previously shared context after each session cannot provide personalized or coherent experiences. This problem of *long-term conversational memory* has attracted growing attention from both academia and industry.

Existing approaches to equipping agents with memory fall into three broad paradigms. **No-memory agents** process each query independently, discarding all historical context—they are cheap and fast but fundamentally incapable of leveraging

past interactions. **Full-context agents** concatenate the entire dialogue history into the prompt; while this preserves complete information, the quadratic cost of attention mechanisms makes this approach prohibitively expensive and slow as conversations grow. **Raw-turn RAG** retrieves relevant dialogue turns from a vector store, reducing context length, but the retrieved turns are noisy, unstructured, and lack the abstraction needed for efficient reasoning.

In this paper, we propose a structured memory pipeline that addresses these limitations. Rather than storing raw dialogue turns, our agent first **extracts** structured facts and detail notes from conversation sessions using a lightweight LLM. These memories are then passed through an **updater** that compares new memories against existing ones, performing deduplication, merging, or replacement. Periodically, a **reflection** module synthesizes high-level abstractions from clusters of active memories—for instance, inferring a user’s personality traits or long-term preferences from accumulated factual observations.

Our key insight is that this *extract–update–reflect* pipeline trades a one-time ingestion cost (memory extraction per session) for drastically reduced inference cost at answer time: only a small set of relevant, structured memories needs to be retrieved and included in the prompt. We evaluate our system against three baselines on a benchmark of 40 questions across four categories, using Qwen2.5-3B as the generation model and DeepSeek-V4-Pro as the LLM judge.

Our contributions are threefold:

1. A new memory agent framework that integrates fact extraction, detail-note preservation, rule-based memory updating, and high-level reflection into a unified, lightweight pipeline.
2. Systematic ablation studies and failure analysis that quantify the contribution of each component (detail notes, updating, reflection) and

identify multi-hop reasoning as the dominant failure mode.

3. A reproducible, open-source benchmark with all code, prompts, and evaluation data publicly available.

2 Related Work

2.1 Conversational Memory and Retrieval-Augmented Generation

The challenge of maintaining state across dialogue turns has been studied extensively. Early work focused on slot-filling dialogue state tracking; more recently, retrieval-augmented generation (RAG; (Lewis et al., 2020)) has emerged as a dominant paradigm for providing external knowledge to LLMs. Systems like LangChain (LangChain Team, 2024) and LlamaIndex (Chase, 2024) offer memory modules that store and retrieve conversation history, but typically operate on raw dialogue turns rather than structured, abstracted memories.

Mem0 (Mem0 Team, 2024) represents a closely related approach, extracting structured memories from conversations and applying deduplication and conflict resolution. Our work builds on similar principles but focuses specifically on the long-term, multi-session setting with an emphasis on lightweight, locally deployable models.

2.2 Long-Term Agent Memory

The concept of long-term memory in AI agents has been popularized by the *Generative Agents* framework (Park et al., 2023), which introduced a memory stream architecture with three key operations: memory retrieval (by recency, importance, and relevance), memory reflection (synthesizing higher-level inferences), and memory planning. Our reflection module is directly inspired by this work, adapted for a simpler, retrieval-based pipeline.

MemoryBank (Zhong et al., 2023) introduced Ebbinghaus-inspired forgetting curves for memory retention, modeling how memories decay over time. While we do not implement forgetting mechanisms, this direction represents a natural extension of our framework.

Other notable approaches include MemGPT (Packer et al., 2023), which treats memory as a virtual context management problem, and ChatDev (Qian et al., 2024), which uses communicative agents with shared memory for software development.

2.3 Memory Update and Reflection

Memory updating—deciding whether to add, merge, or replace existing memories when new information arrives—is critical for preventing memory bloat and contradiction. Rule-based approaches use text similarity thresholds to make these decisions, while LLM-based approaches prompt the model to reason about semantic equivalence. Our system uses rule-based updating for efficiency, with an LLM-based option available for higher precision.

Reflection, as introduced by Generative Agents, involves prompting an LLM to generate abstract, high-level statements given a set of related observations. We apply this technique to clusters of active memories, generating personality summaries, preference patterns, and relational inferences that may not be explicitly stated in any single memory.

2.4 LLM-as-Judge

Using strong language models to evaluate the quality of weaker models’ outputs has become a standard evaluation protocol (Zheng et al., 2023). We adopt this paradigm, using DeepSeek-V4-Pro as a cloud-based judge to score model predictions on a 0–1 correctness scale. This provides a more nuanced evaluation than exact-match metrics alone.

2.5 Positioning

Our work occupies a specific point in the design space: a **low-cost, dynamically maintained memory agent** that targets locally deployable models (3B parameters) and emphasizes structured memory abstraction over raw-turn retrieval. This positions it between heavyweight cloud-based memory systems and retrieval-only baselines.

3 Method

3.1 Overall Architecture

Figure 1 illustrates the complete pipeline of our memory agent. The system operates in two phases: an **ingestion phase** that processes conversation sessions into structured memories, and a **query phase** that retrieves relevant memories to answer questions.

3.2 Writer: Memory Extraction

The Writer module converts raw, multi-turn dialogue into structured memories. Given a conversation session (a sequence of timestamped messages

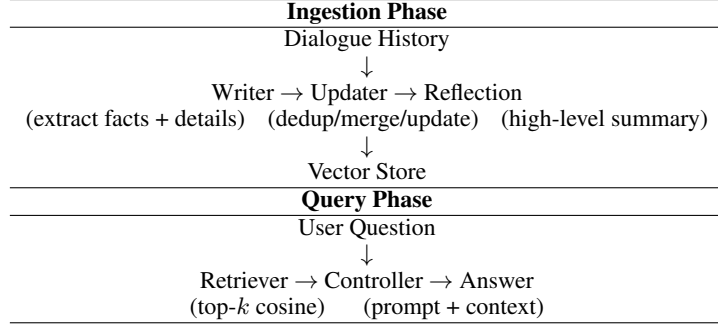


Figure 1: System architecture. During ingestion, each dialogue session passes through the Writer (extracting facts and detail notes), Updater (deduplication and merging), and Reflection (high-level synthesis). The resulting active memories are stored in a vector database. At query time, the Retriever finds the top- k most similar memories, which the Controller combines with the question to generate an answer.

between two speakers), the Writer prompts a language model to extract two types of memories:

- **Facts** (type=fact): Concise, self-contained statements about entities, events, relationships, preferences, and other information discussed in the conversation. Each fact is timestamped and attributed to the speaker who stated it.
- **Detail Notes** (type=detail): Specific, granular pieces of information that are often lost in summarization—dates, book titles, food items, vehicle models, numerical values, and location names. These are stored separately because they are critical for answering detail-oriented questions but tend to be omitted when the model summarizes at a higher level.

The extraction prompt instructs the model to produce a fixed number of fact memories per session (default $m = 4$) and to include detail notes when the MEMORY_DETAIL_NOTES flag is enabled. Each memory is stored as a JSON object with fields for type, content, timestamp, and speaker.

3.3 Updater: Memory Management

As new conversation sessions are ingested, the Updater compares each newly extracted memory against the existing memory store to prevent duplication and manage contradictions. For each new memory, the system retrieves the top existing memory by cosine similarity and applies one of four actions:

- **ADD**: The new memory is sufficiently distinct from all existing memories and is inserted as a new active memory.

- **IGNORE**: The new memory is semantically equivalent to an existing memory (cosine similarity above the merge threshold) and is discarded.
- **MERGE**: The new memory is similar to but not identical to an existing memory; the existing memory’s content is enriched with the new information.
- **UPDATE**: The new memory contradicts an existing memory; the old memory is marked as obsolete and replaced. (This action is only available with LLM-based updating enabled.)

In our default configuration, we use a rule-based updater (MEMORY_UPDATE_USE_LLM=0) that relies on cosine similarity thresholds. In our small-sample experiments, this resulted in 854 extracted memories being reduced to 677 active memories (177 duplicates merged or ignored). The most frequent action was MERGE (169 occurrences), indicating that the conversations contained many overlapping or progressively refined statements rather than direct contradictions.

3.4 Reflection: High-Level Memory Synthesis

Inspired by the reflection mechanism in Generative Agents (Park et al., 2023), our Reflection module periodically synthesizes higher-level abstractions from the accumulated active memories. Given a set of active memories (sorted by recency, with a configurable count limit), the module prompts the LLM to generate a small number of reflective statements—for example, inferring personality traits, relationship dynamics, or long-term behavioral patterns.

In our experiments, 5 conversation sessions produced 16 reflection memories (with

MEMORY_REFLECTIONS=6). These reflections are stored alongside regular fact and detail memories in the vector store and are retrieved and used in the same way during query time.

3.5 Retriever and Controller

Retriever: At query time, the user’s question is embedded using the same embedding model used during ingestion (all-MiniLM-L6-v2). We retrieve the top- k most similar memories by cosine similarity (default $k = 10$).

Controller: The Controller constructs a prompt that includes: (1) the user’s current question, (2) the retrieved memories (formatted with type, timestamp, speaker, and content), and (3) an instruction to answer concisely and to say “unknown” if the memories do not contain sufficient information. The prompt is then sent to the generation model, and the response is returned as the agent’s answer.

3.6 Baselines

We compare our system against three baselines:

- **No-Memory:** The agent receives only the current question, with no access to any dialogue history.
- **Full-Context:** All dialogue sessions (272 turns across 10 conversations) are concatenated and included in the prompt.
- **Raw-Turn RAG:** Individual dialogue turns are embedded and stored in a vector database. At query time, the top- k most similar turns are retrieved and included in the prompt.

4 Experiments

4.1 Experimental Setup

Dataset: We use the *p10* evaluation set, generated from the LoCoMo benchmark (Maharana et al., 2024) with `-per_category 10 -seed 42`. The set contains 10 conversations with a total of 272 dialogue sessions, from which 40 questions are drawn. Questions are evenly distributed across four categories, as shown in Table 1.

Models: All systems use **Qwen2.5-3B-Instruct** (Qwen Team, 2024) (running locally via Ollama) as the generation model for both memory extraction and answer generation. Embeddings are computed using all-MiniLM-L6-v2 (Reimers and Gurevych, 2019). For evaluation, we use **DeepSeek-V4-Pro** (accessed via SiliconFlow cloud API) as the LLM judge.

Metrics: We report four metrics:

- **Judge Score:** A continuous 0–1 score assigned by the LLM judge (0 = completely wrong, 0.5 = partially correct, 1 = fully correct).
- **F1:** Token-level F1 between the predicted and reference answers.
- **EM:** Exact match accuracy.
- **Avg. Latency:** Wall-clock time per question, including retrieval and generation.

Hyperparameters: Memory extraction uses $m = 4$ facts per session with detail notes enabled. Retrieval uses $k = 10$ and pure cosine similarity. The updater uses rule-based merging with a cosine similarity threshold of 0.85. Reflection generates up to 6 high-level statements per conversation.

4.2 Main Results

Table 2 presents the overall comparison of all four systems on the p10 benchmark.

Our Update+Reflection system achieves a Judge Score of 0.325, outperforming Raw-Turn RAG by 23.6% ($0.263 \rightarrow 0.325$) and substantially closing the gap to Full-Context (0.400). In terms of efficiency, our system answers in 1.22 seconds on average— $6\times$ faster than Full-Context (7.31s)—while maintaining answer quality far above the No-Memory baseline.

The latency advantage is structural: Full-Context must process all 272 dialogue turns for every question, whereas our system pays a one-time ingestion cost (memory extraction at session boundaries) and then only retrieves 10 compact memory items per query. The ingestion cost is amortized across all questions asked about a conversation.

4.3 Performance by Question Category

Table 3 breaks down Judge Scores by question category.

Our system performs best on temporal questions (0.500), where structured memory timestamps likely provide an advantage over raw-turn retrieval. On open-domain questions, Full-Context dominates (0.800), suggesting that some broad knowledge questions benefit from the full conversational context that our compressed memories cannot fully capture.

The most striking pattern is the difficulty all systems have with **multi-hop questions**: even

Category	Description	Count
temporal	Time-related reasoning	10
open_domain	Broad knowledge about characters	10
single_hop	Answerable from a single fact	10
multi_hop	Requires combining ≥ 2 facts	10

Table 1: Question categories in the p10 evaluation set.

System	Judge Score $\uparrow$	F1 $\uparrow$	EM $\uparrow$	Latency (s) $\downarrow$
No-Memory	0.000	0.000	0.00	0.11
Full-Context	0.400	0.154	0.05	7.31
Raw-Turn RAG	0.263	0.073	0.00	0.79
Update+Reflection (Ours)	0.325	0.119	0.00	1.22

Table 2: Main results on the p10 benchmark (40 questions). Best results in each column are **bolded**.

Full-Context only achieves 0.300, Raw-Turn RAG scores 0.000, and our system scores 0.100. This reveals multi-hop reasoning as a fundamental bottleneck for retrieval-based memory systems.

4.4 Ablation Study

To quantify the contribution of each component, we conducted ablation experiments on a smaller 8-question evaluation set using the same Qwen2.5-3B model. Table 4 reports token-level F1 scores.

Key findings from the ablation study:

1. **Detail notes provide the largest single improvement** (+0.073 F1 over append-only). Preserving concrete details (dates, names, numbers) that are often lost in summarization significantly boosts retrieval quality for fact-seeking questions.
2. **Rule-based updating slightly reduces F1** (0.304 $\rightarrow$ 0.289) while reducing memory count from 854 to 677 (a 20.7% reduction). This trade-off—slightly lower quality for a leaner memory store—may be acceptable in storage-constrained deployments.
3. **Reflection shows no measurable improvement** on this evaluation set (0.289 $\rightarrow$ 0.289). The small-sample questions primarily require specific facts rather than high-level personality inferences. We expect reflection to become more valuable on larger datasets with questions about character traits and relationship dynamics.
4. **More memories hurt performance:** increasing from $m = 4$ to $m = 8$ facts per session caused F1 to drop from 0.231 to 0.101. This

confirms that retrieval noise can outweigh the benefit of more candidate memories.

5. **Cautious inference** (allowing the model to use strong indirect evidence for “likely/probably” answers) reduced the unknown rate from 4 to 3 but also reduced F1 from 0.289 to 0.275, suggesting a trade-off between coverage and precision.

4.5 Efficiency Analysis

Table 5 compares the computational cost of each system.

While our system makes more total LLM calls (322 vs. 40 for baselines), the vast majority (272) occur during the one-time ingestion phase. At query time, only one LLM call is needed (for answer generation), plus the cost of embedding-based retrieval. The per-question answer latency (1.22s) is only slightly higher than Raw-Turn RAG (0.79s) and dramatically lower than Full-Context (7.31s).

The ingestion cost breakdown is: 272 calls for memory extraction (one per session), 10 calls for reflection (one per conversation), and 40 calls for answer generation. In a production setting where one conversation is ingested once but queried many times, the amortized cost per question approaches 1 LLM call.

4.6 Failure Analysis

To understand the limitations of our system, we analyzed cases where the Judge Score was 0 (completely wrong). We present three representative failure modes below.

4.6.1 Case 1: Multi-hop inference failure

Question: *Would Caroline pursue writing as a career option?*

System	Temporal	Open-Domain	Single-Hop	Multi-Hop
No-Memory	0.000	0.000	0.000	0.000
Full-Context	0.200	0.800	0.300	0.300
Raw-Turn RAG	0.250	0.600	0.200	0.000
Update+Reflection	0.500	0.500	0.200	0.100

Table 3: Judge Score by question category.

Configuration	F1
Append-only ($m=4, k=6$)	0.231
+ Detail notes	0.304
+ Update (rule-based)	0.289
+ Reflection	0.289
+ Cautious inference prompt	0.275
Append-only ($m=8, k=10$)	0.101

Table 4: Ablation results on the small evaluation set (8 questions).

Reference: Likely no; though she likes reading, she wants to be a counselor.

Our answer: unknown

Diagnosis: The question requires connecting Caroline’s stated career preference (counseling) with the absence of evidence for writing as a career—a classic multi-hop inference. The system retrieves the right memories (counseling interest) but the generation model (Qwen2.5-3B) fails to perform the counterfactual reasoning needed to conclude she would *not* pursue writing, and defaults to “unknown.”

4.6.2 Case 2: Missing bridging fact

Question: *What console does Nate own?*

Reference: A Nintendo Switch; since the game “Xenoblade 2” is made for this console.

Our answer: unknown

Diagnosis: The bridging fact (Nate plays “Xenoblade 2” $\rightarrow$ therefore owns a Nintendo Switch) was either not extracted during the writing phase or was ranked below the top-10 retrieval cutoff. This highlights a retrieval recall problem: the necessary fact exists in the dialogue but does not surface in the top- k results.

4.6.3 Case 3: Entity confusion

Question: *Would Melanie be considered a member of the LGBTQ community?*

Reference: Likely no, she does not refer to herself as part of it.

Our answer: unknown

Diagnosis: The system retrieves highly relevant but *wrong-entity* memories. The embedding model conflates the topic (LGBTQ) with the person (Caroline vs. Melanie). This entity confusion is a known weakness of embedding-based retrieval and suggests the need for entity-aware filtering or metadata-based re-ranking.

Summary of failure patterns. Table 6 summarizes the observed failure modes.

The dominant failure mode—multi-hop reasoning—affects all retrieval-based systems including Full-Context (which also scores only 0.300 on multi-hop questions). This suggests that the bottleneck is not in the memory architecture per se, but in the reasoning capability of the 3B-parameter generation model.

5 Discussion

5.1 Why Does Full-Context Still Win?

Despite our system’s improvements over Raw-Turn RAG, Full-Context maintains a clear lead (0.400 vs. 0.325 Judge Score). This gap reflects the fundamental trade-off between **information completeness** and **computational efficiency**. Full-Context preserves every detail of every conversation—nuance, tone, implied relationships, and subtle context that are inevitably lost during memory extraction. Our structured memories are lossy by design: they capture the “gist” but not the full richness of dialogue.

System	Total LLM Calls	Calls/Question	Answer Latency (s)
No-Memory	40	1.0	0.11
Full-Context	40	1.0	7.31
Raw-Turn RAG	40	1.0	0.79
Update+Reflection	322	8.05	1.22

Table 5: Computational cost comparison (p10, 40 questions).

Pattern	Frequency	Root Cause
Multi-hop reasoning failure	High	Model cannot combine retrieved facts
Missing bridging fact	Medium	Retrieval recall gap; fact not in top- k
Entity confusion	Medium	Embedding similarity conflates entities
Conservative “unknown”	Medium	Model defaults to “unknown” when uncertain
Memory compression loss	Low	Important information lost during extraction

Table 6: Summary of failure patterns.

However, the efficiency advantage of our system is substantial ($6\times$ faster) and grows linearly with conversation length. For applications where sub-second response times matter more than perfect accuracy—personal assistants, customer support chatbots, real-time game NPCs—our approach offers a pragmatic middle ground.

5.2 When Is Our Approach Preferable?

Our system is most advantageous when latency matters (1.22s vs. 7.31s), storage is constrained, privacy is a concern (extracted memories abstract away raw conversation text), or multiple queries per conversation amortize the ingestion cost. It is less suitable when perfect recall is required, multi-hop reasoning dominates, or nuance and subtext matter.

5.3 The Multi-Hop Challenge

Multi-hop reasoning emerges as the clearest limitation of our system and of retrieval-based memory approaches in general. All systems—including Full-Context—perform poorly on multi-hop questions (max Judge Score 0.300). We believe this requires not just better retrieval but fundamentally different architectures: (1) **multi-step retrieval**, iteratively retrieving new memories conditioned on previous results; (2) **graph-structured memory**, representing memories as entities and relationships rather than flat text chunks; and (3) **stronger generation models**, as the 3B model may simply lack the reasoning capacity for complex multi-hop inference.

5.4 Limitations

Beyond multi-hop reasoning, our work has several limitations: the p10 benchmark (40 questions) is sufficient for ablation studies but larger-scale validation is needed; results are based on a 3B-parameter model and performance with larger models remains untested; our experiments did not demonstrate clear benefits from reflection, likely because the dataset focuses on factoid questions; and unlike MemoryBank, our system has no mechanism for active forgetting.

6 Conclusion

We presented a lightweight long-term memory agent that extracts structured facts and detail notes from dialogue, applies rule-based updating to manage memory growth, and generates high-level reflections. On a 40-question benchmark, our Update+Reflection system achieves a Judge Score of 0.325—a 23.6% improvement over raw-turn RAG (0.263)—while answering $6\times$ faster than Full-Context (1.22s vs. 7.31s).

Ablation confirms that detail notes and memory updating are the most impactful components; reflection’s benefits are context-dependent. Failure analysis identifies multi-hop reasoning as the primary bottleneck, pointing to the need for multi-step retrieval or graph-based memory.

Our code and data are publicly available at <https://github.com/kkSaMa-123/nlp>. We hope this work provides a useful baseline for future research on efficient, structured long-term memory for dialogue agents.

References

- Harrison Chase. 2024. LlamaIndex: A data framework for LLM applications. https://github.com/run-llama/llama_index.
- LangChain Team. 2024. LangChain: Build context-aware reasoning applications. <https://github.com/langchain-ai/langchain>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Mem0 Team. 2024. Mem0: The memory layer for personalized AI. <https://github.com/mem0ai/mem0>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulators of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.
- Chen Qian, Xin Cong, Wei Liu, Cheng Yang, Yidong Dang, Jie Li, Zhiyuan Liu, and Maosong Sun. 2024. Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Qwen Team. 2024. Qwen2.5: A party of foundation models. *arXiv preprint arXiv:2412.15115*.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*.

A Prompt Templates

A.1 Memory Writer Prompt (simplified)

You are a memory extraction assistant. Given a conversation between two speakers, extract the most important factual information as structured memories.

For each memory, output a JSON object with:

- type: "fact" or "detail"
- content: a concise, self-contained statement
- timestamp: the time of the message
- speaker: the person who stated this information

Focus on: personal information, preferences, life events, relationships, plans, opinions, and concrete details (dates, names, numbers, locations).

Output exactly {m} fact memories and any relevant detail notes.

A.2 Updater Prompt (LLM-based mode)

You are a memory management assistant. Compare the NEW memory against the EXISTING memory and decide:

- ADD: new memory contains new information not in existing memory
- IGNORE: new memory is already fully covered by existing memory
- MERGE: combine overlapping information from both memories
- UPDATE: new memory contradicts existing memory; keep the new one

Existing memory: {existing_memory}

New memory: {new_memory}

Decision (ADD/IGNORE/MERGE/UPDATE):

Merged or updated content (if applicable):

A.3 Controller Answer Prompt (simplified)

You are a helpful assistant with access to conversation memories. Answer the question based ONLY on the retrieved memories below.

If the memories contain enough information, answer concisely.

If the memories do not contain the answer, say "unknown".

Retrieved Memories:

{retrieved_memories}

Question: {question}

Answer:

B Hyperparameter Configuration

Parameter	Value
Generation model	Qwen2.5-3B-Instruct
Embedding model	all-MiniLM-L6-v2
Judge model	DeepSeek-V4-Pro
MEMORY_PER_SESSION (m)	4
MEMORY_TOP_K (k)	10
MEMORY_DETAIL_NOTES	1 (enabled)
MEMORY_UPDATE_USE_LLM	0 (rule-based)
MEMORY_REFLECTIONS	6
MEMORY_ALLOW_INFERENCE	0 (disabled)
Cosine similarity threshold (merge)	0.85
Embedding dimension	384

C Code Repository

Key entry points in the open-source release:

- `memory_agent/eval/run_eval.py` — Main evaluation runner
- `experiments/run_p10_experiments.sh` — p10 prediction experiments
- `experiments/run_p10_cloud_judge.sh` — Cloud LLM judge evaluation
- `experiments/run_small_experiments.sh` — Small-set ablation experiments